

TOOL

Anonymous Author(s)*

ABSTRACT

1 INTRODUCTION

In recent years, the open-source software (OSS) ecosystem has witnessed substantial growth [? ?], providing developers with a vast array of options for using free software. It has become a common practice for developers to adopt third-party OSS components to accelerate development and reduce costs [?]. Consequently, the vulnerability landscape has expanded beyond traditional proprietary software to encompass open-source software (OSS). The increasing vulnerabilities in open-source software have raised significant concerns, particularly regarding supply chain attacks [?]. Attackers can exploit these vulnerabilities, affecting not only the open-source software itself but also its downstream users, thus amplifying potential financial gains and cyber chaos [?].

Notably, downstream software shares or reuse logic from upstream OSS components. As these OSS components evolve and downstream software undergoes customization, the code initially derived from OSS components diverges from the original one at both ends. Consequently, it leads to challenges in mitigating *Vulnerable Code Clones* (VCCs) [?]. When facing incompatible changes or customized OSS components, mitigating VCCs by replacing newer OSS components becomes infeasible or cumbersome, especially in languages like C/C++ that lacks a dominant package manager.

In this context, developers often resort to manually finding and backporting security patches into downstream software. However, it is time-consuming and resource-intensive. Notably, 80% of attacks leveraged the vulnerabilities reported more than three years ago [?], indicating that the issue of VCCs has not been fully resolved.

Typically, developers employ vulnerable code clone (VCC) discovery techniques [? ? ? ?] to identify reused components (such as files, functions, or fragments) in a target project program. They can also use learning-based methods for classifying vulnerable functions [? ? ? ?]. However, the effectiveness of vulnerability detection is often hindered in cases of *Vulnerabilities with Multiple Fixing Functions* (VM), where fixing patches span across multiple functions or files. Unfortunately, VM are inevitably prevalent in real world due to complex inter-procedural program logic, posing challenges for accurate VM detection.

Limitations of Existing Approaches. Existing research often addresses the detection of *Vulnerabilities with Multiple Fixing Functions* (VM) by dividing the task into matching vulnerability characteristics within individual functions (*i.e.*, matching-one-in-all approach). Clone-based techniques [? ? ? ?] identify vulnerable code clones (VCCs) within functions, while learning-based approaches represent functions in abstract spaces and classify them as vulnerable or not [? ? ? ?]. However, these approaches assume equal importance for all fixing functions in a vulnerability, potentially leading to over-representation resulting in false alarms. Additionally, while software composition analysis (SCA) techniques [? ? ? ?] can detect VM, they are primarily designed for identifying reused OSS components, which may overlook specific vulnerability or fixing characteristics, resulting in false alarms.

Challenge. Naturally thinking, assigning all fixing equal importance is straightforward and may not accurately represent the actual characteristics of vulnerabilities. Some fixing functions may carry adequate information to represent a vulnerability, while others may not. However, it is challenging to measure and weigh the unique fingerprints of each fixing function to a vulnerability. The primary challenge lies in how to capture and distinguish the uniqueness of fixing functions and identify the critical ones for a vulnerability.

Our Approach. To tackle this challenge, we introduce TOOL, a novel approach for Vulnerabilities-with-Multiple-Fixing-Function-Detection. Unlike traditional methods that treat all fixing functions equally, TOOL prioritizes the fixing functions and selects the most critical functions for vulnerability representation (see Section ??). It involves matching the signatures [? ?] of these critical functions. To cope with the potential decrease in recall due to excluding the remaining fixing functions, we employ semantic equivalent statement matching, which consists of program rephrasing [?] (see Section ??) and contextual semantic equivalent statement mapping (see Section ??). The program rephrasing aims to uncover more VM by creating another signature for the rephrased form of the critical functions, while contextual semantic equivalent statement mapping aims to match the composing statements precisely based on their contextual semantic equivalence.

The insight of the contextual semantic equivalent statement mapping stems from our observation that some syntactically identical statements serve different contextual purposes. Existing approaches add new context similarity constraints using syntactically equivalent statements, which does not correct the error from the essential. We claim that “statement equivalence” should be redefined not only from syntactic similarity, but also from their contextual similarity. To our best knowledge, TOOL is the first to discriminate critical functions from fixing functions and consider both program rephrasing and contextual semantic equivalent statements to identify vulnerabilities with high quantity and quality.

Evaluation. We implemented TOOL and generated vulnerability signatures from 810 CVEs. We conducted an evaluation using a dataset comprised of the top 1,000 C/C++ real-world projects selected from GitHub. Comparing TOOL with state-of-the-art techniques, TOOL achieves a f1-score of 0.84, outperforming the best state-of-the-art at 17.6%. TOOL discovered 275 VM across 104 projects, with 42 of them confirmed by developers and 5 of them assigned with corresponding new CVE identifiers. We observed the robustness of TOOL regarding the fixing function number in the VM. We conducted an ablation study to observe the contribution of key components in TOOL. We also measured the threshold sensitivity and performance of TOOL. The results demonstrate TOOL’s effectiveness and efficiency in identifying VM within real-world projects.

Contribution. We summarize our contributions as follows:

- We propose TOOL, a novel approach to detect Vulnerabilities with Multiple Fixing Functions (VM) in real-world projects.
- We evaluate TOOL on our ground truth collected from real-world C/C++ projects and demonstrate its effectiveness compared with the state-of-the-art approaches.

- TOOL has discovered 275 vulnerabilities in 84 C/C++ projects. 42 are confirmed by developers, 5 has been assigned with CVE identifiers, demonstrating its effectiveness in real-world projects.

2 EMPIRICAL STUDY

3 DESIGN OF TOOL

In this section, we present the design of TOOL.

4 EVALUATION

We evaluate TOOL based on the following research questions.

- **RQ1. Effectiveness Evaluation.** How is the effectiveness of TOOL compared to the state-of-the-art tools?
- **RQ2. Robustness Evaluation.** How is the robustness TOOL in detecting VM?
- **RQ3. Ablation Study.** How does each component contribute to TOOL's effectiveness?
- **RQ4. Threshold Sensitivity.** How do the thresholds contribute to the effectiveness of TOOL?
- **RQ5. Performance Evaluation.** How is the efficiency of TOOL?

4.1 Setup

4.2 Discussion

Threats. First,

Limitations. First,

5 RELATED WORK

We review previous work related to

6 CONCLUSIONS

In this paper,

A EFFECTIVENESS RESULTS ON TARGET PROJECTS OF TOOL

We present true positives, false positives, detected by TOOL and the false negatives of TOOL in Table ??, The project identifier used in Table ?? corresponds to the project names listed in Table ??.

Table 1: The Number of True Positives, False positives, Detected by tool and the False Negatives tool Missed

Proj	TP (#)	FP (#)	FN (#)	Proj	TP (#)	FP (#)	FN (#)	Proj	TP (#)	FP (#)	FN (#)	Proj	TP (#)	FP (#)	FN (#)
1	25	2	4	27	1	0	0	53	2	0	3	79	1	0	0
2	4	0	0	28	3	0	0	54	1	0	0	80	1	0	0
3	26	2	5	29	1	0	0	55	2	0	0	81	1	0	0
4	4	0	2	30	2	0	0	56	1	0	0	82	1	0	0
5	1	1	0	31	1	0	0	57	0	1	0	83	1	0	0
6	20	1	7	32	1	0	0	58	0	1	0	84	1	0	0
7	4	0	1	33	1	0	0	59	1	0	0	85	1	0	0
8	6	0	0	34	8	1	0	60	1	0	0	86	1	0	0
9	3	1	1	35	1	0	0	61	6	0	1	87	1	0	0
10	3	0	0	36	0	1	0	62	1	0	0	88	0	1	0
11	1	0	0	37	17	0	4	63	1	0	0	89	0	1	0
12	2	0	1	38	0	8	0	64	0	4	0	90	1	0	0
13	4	1	0	39	2	0	0	65	2	0	0	91	1	0	0
14	8	0	3	40	1	0	0	66	2	0	0	92	1	0	0
15	10	1	6	41	2	0	0	67	0	3	0	93	0	1	0
16	3	0	0	42	0	8	0	68	1	1	0	94	1	0	0
17	5	1	5	43	1	0	0	69	0	1	0	95	1	0	0
18	2	0	0	44	1	0	0	70	0	1	0	96	1	0	0
19	2	0	0	45	2	0	3	71	1	0	0	97	0	0	1
20	0	1	1	46	1	0	0	72	0	1	0	98	1	0	0
21	1	0	0	47	1	0	0	73	1	0	0	99	1	0	0
22	4	0	0	48	1	0	0	74	1	1	0	100	0	1	0
23	1	1	0	49	1	0	0	75	1	0	0	101	1	0	0
24	3	0	1	50	1	0	0	76	2	0	0	102	0	0	1
25	1	0	0	51	23	1	4	77	1	0	0	103	1	0	0
26	2	0	0	52	8	1	0	78	1	1	0	104	1	0	0

Table 2: Projects Id. and Project Names

Id.	Project Name	Id.	Project Name	Id.	Project Name	Id.	Project Name
1	hanwckf/rt-n56u	27	rogerclarkmelbourne/Arduino_STM32	53	huangz1990/annotated_redis_source	79	wireshark/wireshark
2	ravynsoft/ravynos	28	rmtheis/tess-two	54	OpenAtomFoundation/TencentOS-tiny	80	ejoy/ejoy2d
3	Stichting-MINIX-Research-Foundation/minix	29	eduard-permyakov/permafrost-engine	55	NetBSD/src	81	jart/cosmopolitan
4	seemoo-lab/nexmon	30	leixiaohua1020/simplest_ffmpeg_mobile	56	sdlpal/sdlpal	82	arendst/Tasmota
5	alibaba/tengine	31	peng-zhihui/Dummy-Robot	57	b4winckler/macvim	83	flipperdevices/flipperzero-firmware
6	fastos/fastsocket	32	armink/EasyLogger	58	veracrypt/VeraCrypt	84	nmap/nmap
7	panda-re/panda	33	Ralim/IronOS	59	openzfs/zfs	85	RedisGraph/RedisGraph
8	tanersener/mobile-ffmpeg	34	kbengine/kbengine	60	antirez/disque	86	roflor/proxychains-ng
9	premake/premake-core	35	endrazine/wcc	61	rizinorg/rizin	87	darktable-org/darktable
10	civetweb/civetweb	36	flatpak/flatpak	62	reactos/reactos	88	antirez/sds
11	madeye/proxydroid	37	AdAway/AdAway	63	nicolasff/webdis	89	openssl/openssl
12	y123456yz/reading-code-of-nginx-1.9.2	38	microsoft/WSL-Linux-Kernel	64	FreeRDP/FreeRDP	90	mabeijianxi/small-video-record
13	freebsd/freebsd-src	39	catboost/catboost	65	radareorg/radare2	91	espressif/ESP8266_RTOS_SDK
14	CloverHackyColor/CloverBootloader	40	mridgers/clink	66	ApsaraDB/PolarDB-for-PostgreSQL	92	ctfs/write-ups-2016
15	damonkohler/sl4a	41	gpac/gpac	67	swaywm/sway	93	nginx/nginx
16	joncampbell123/dosbox-x	42	spotify/linux	68	mjolnirapp/mjolnir	94	jiangdongguo/AndroidUSBCamera
17	alibaba/AliOS-Things	43	emscripten-core/emscripten	69	greenplum-db/gpdb	95	ntop/PF_RING
18	zlgopen/awtk	44	Sunzxyong/Tiny	70	systemd/systemd	96	h2o/h2o
19	sumatrapdfreader/sumatrapdf	45	huangz1990/redis-3.0-annotated	71	RediSearch/RediSearch	97	gozfrees/gear-lib
20	nginx/nginx-releases	46	lavabit/magma	72	ultrajson/ultrajson	98	omnirom/android_bootable_recovery
21	peng-zhihui/ElectronBot	47	ossec/ossec-hids	73	slact/nchan	99	TeamWin/Team-Win-Recovery-Project
22	TelegramMessenger/Telegram-iOS	48	moby/hyperkit	74	ImageMagick/ImageMagick	100	coolwanglu/vim.js
23	Tencent/xLua	49	Proxmark/proxmark3	75	momotech/MLN	101	vifm/vifm
24	Cisco-Talos/pyrebox	50	cesanta/mongoose	76	nodemcu/nodemcu-firmware	102	matttrajca/sudo-touchid
25	aws/amazon-freertos	51	alibaba/LVS	77	saki4510t/UVCCamera	103	micropython/micropython
26	yangchaojiang/yjPlay	52	F-Stack/f-stack	78	libretro/RetroArch	104	InfiniTimeOrg/InfiniTime

Temporary page!

$\LaTeX$ was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because $\LaTeX$ now knows how many pages to expect for this document.